

HANDBOOK

Broadside

Help and reference

Every tab of Broadside, control by control: what it does, what it will ask you first, and what it will not do behind your back. With a glossary for the terms and two reference tables for the keys and the warnings.

Version 1.3.0-build40-3 · August 2026

Contents

1 Overview and getting started

2 Hosts

3 Terminals

4 Broadcast

5 PTY Broadcast

6 MultiTerminal

7 Skills

8 SFTP

9 Logs

10 Settings

11 Security

12 System requirements

13 Keyboard shortcuts

14 Warnings and messages

15 Glossary

This is the same help the app carries, printed as it stood at version 1.3.0-build40-3. The copy inside Broadside is the one that is kept up to date: open **Help** in the app if this document is older than the version you are running.

SECTION 1

Overview and getting started

Broadside is an SSH console built around one idea: send one command to many machines at once, while still giving each machine its own full interactive terminal when you need it. It also opens local Windows shells in the same window.

The left rail

The buttons down the left side switch between the main areas of the app: **Hosts**, **Terminals**, **Broadcast**, **PTY Broadcast**, **MultiTerminal**, **Skills**, **SFTP**, **Logs** and **Settings**. **Help** and **About** sit at the bottom. The rail collapses to icons only on narrow windows, or with the collapse button at the top. Drag its right edge to resize it, or double-click that edge to reset the width.

Finding your way

- Hover almost any control and a short explanation appears in the status bar along the bottom of the window. The right-hand end of that bar counts what the page is showing. On **Terminals** that is how many tabs are open, and it becomes a pair like **16/20 sessions connected** as soon as one of them is not: the second number is the tabs, the first is the ones with a live connection, and the difference is the tabs showing red.
- The current app version is shown next to the Broadside name at the top of the rail and in the About dialog.
- Switch between light and dark appearance in **Settings**, under **Appearance**.
- Press **F11** (or the maximize button on a terminal tab) to fill the whole window with a single terminal, and **F11** again to return.

SECTION 2

Hosts

The **Hosts** tab is your address book of machines to connect to. Each host records how to reach it (address, port, username) and how to sign in.

Adding and editing hosts

- Give each host a label, address, port and username. You can assign a color so the host is easy to spot across the app.
- Choose how to sign in: a password or an SSH key (with an optional passphrase). You can also store a sudo password for administrator commands.
- For your safety, saved passwords and passphrases are never shown again after you save them. You can replace a secret, but the app will not display the stored value. See the **Security section** for where these are kept.
- A host row can carry a small amber chip — most often the name of a login shell Broadside cannot fully drive. **Warnings and messages** lists every chip, banner and error the app can show you and what each one actually means.

Importing many hosts

Use **Import** to load a list of hosts from a csv file. The importer only reads connection details, never passwords. If a label is already in use for a different machine, the import keeps both by adding a numbered suffix (for example web becomes web-2).

Tip: to build that file without guessing the format, add one host here manually first, use **Export** to save the list to a CSV or spreadsheet, then fill in the rest of your hosts in that same file (keeping its column headers) and **Import** it back.

Working with the list

- The **Status** column shows whether each host currently answers on the network. Its header is the single letter S when the column is narrow.
- A colored dot marks hosts that have a live terminal connection open right now.
- Select several hosts and open a terminal for all of them at once.
- The **Find by label…** box at the top right narrows the table to hosts whose label matches what you type (case-insensitive), the same as the search boxes on the broadcast rails. It combines with the tag filter and clears with the **x**.
- You can hide columns you do not use from **Settings**, under **Appearance**. When a column is hidden, each row gets a small tag/label icon in its **Actions**; click it to see that host's values for the hidden columns.

Tags

- Give a host one or more tags to group machines (for example prod, web, eu). Separate multiple tags with commas. Each tag shows as its own small chip in the **Tag** column; untagged hosts show a dash. As you type a tag on the host form, a dropdown suggests matching tags already used elsewhere, so spellings stay consistent.
- Click the **Tag** header to sort by the host's first tag (untagged hosts sink to the bottom, or rise to the top when you reverse the sort).
- The filter icon in the **Tag** header opens a checklist of every tag in use. Uncheck tags to hide the hosts that only carry them; a host stays visible as long as one of its tags is still checked. The icon is highlighted while a filter is active, and the status bar shows how many hosts are being shown. The filter lasts for the session and resets when you restart the app.
- Drag the separator on the right edge of the **Tag** column to resize it, or double-click that edge to fit it to the contents. The width is remembered.

SECTION 3

Terminals

Terminals gives you full, interactive terminal sessions, one per tab. Each tab is a real shell running over a PTY, so editors, progress bars and anything that needs a live screen all work normally.

Remote and local shells

- Open a remote shell by starting a terminal for a host from the **Hosts** tab. The connection runs over SSH.
- Open a local Windows shell with the **plus button** at the end of the tab strip. It lists the shells installed on your PC: Windows PowerShell, PowerShell 7 (pwsh), Command Prompt and any installed WSL distributions. Local shells run through ConPTY.
- Rename a local shell tab with the **pencil** on the tab: type a name and press **Enter** (or **Esc** to cancel). The name is used everywhere that tab appears, including the grid tile and the broadcast lists, and an empty name puts the shell's own name back. The rename lasts for the rest of the session and is not kept across restarts. Tabs opened from a profile already carry the profile's name, so they have no pencil.
- A second tab on the same shell or the same host is numbered: **PowerShell**, then **PowerShell 02**, **PowerShell 03**. The number counts the order you opened them in, not the order they sit in, so dragging a tab about never renames anything. A tab you have renamed keeps its number without showing it, which is why renaming the second one leaves you with **PowerShell**, your own name, and **PowerShell 03**: renaming one tab is not a reason to relabel a different one under you.

How local shells are detected (important)

- Detection uses your Windows PATH, not a fixed install folder, so the install location does not matter as long as the installer added the shell to PATH. A shell installed without being put on PATH (for example an unzipped portable build, or a disabled app execution alias) will not appear in the list.
- Broadside reads PATH when it starts. If you install a new shell while the app is already running, restart Broadside for it to show up in the launcher.
- You can hide shells you do not use from the launcher in **Settings** under **Appearance**. Every detected shell is shown by default; a newly installed one appears automatically once detected.

Your own terminal applications

- Detection only finds the shells Broadside knows how to look for. For anything else, such as **Git Bash**, MSYS2 or Cygwin, open **Settings** and the **Custom terminals** section and add it yourself. There is an **Add Git Bash** button that fills in the usual path and options for you, so that case is normally one click and a save.

- Give it a name (used in the launcher and as the tab title), the full path to the program (the **Browse** button finds it), any arguments it needs, and which command language it speaks. Git Bash is normally given `--login -i`. It opens without those, because a program started in a terminal tab already has a screen and a keyboard and Bash goes interactive on its own; what `--login` adds is the profile Git for Windows ships, which is where the coloured `MINGW64` prompt with the branch name and the full set of Unix tools on the path come from. Without it you get a working but plain `bash-5.2$` shell.
- The program must be a full path to a real `.exe`. Broadside checks that when you save, so a typo is caught there rather than when you try to open a tab. Arguments are handed to the program directly rather than through a shell, so there is no quoting to get right.
- The command language matters for more than the icon: it decides which **PTY Broadcast** tab the terminal belongs to, so a broadcast command is always valid for everything it reaches. A Bash-like (posix) terminal joins the **WSL / POSIX** tab.
- A terminal you add appears in the **+** menu straight away, with no restart, and can be used by a saved shell profile like any other shell. Removing one takes it out of the launcher; a profile still pointing at it is flagged rather than silently broken.

Shell profiles

- A profile is a saved local shell that opens the way you want it: a chosen shell, a working directory, and an optional command that runs the moment the shell is ready. Open the **plus button** and use the **Profiles** section at the bottom of the menu to create, launch, edit or delete them.
- **New** opens a short form. Give the profile a name, pick the shell, choose the **Working directory** (type it, or use **Browse** to pick a folder), and optionally set a **Startup command**. The name becomes the tab label, so a profile tab is easy to tell apart from a plain shell tab. Leave the working directory empty to start in your home folder, the same as a plain shell.
- Launching a profile opens the shell in that folder and, a moment later, types the startup command for you and presses Enter, exactly as if you had typed it. The short pause lets the shell finish drawing its prompt first. As one example, a profile can open PowerShell 7 in a project folder and start a command-line tool there in a single click.
- Profiles honour the **How many** count, so you can open several copies of the same profile at once, and a large batch asks first. A profile with a startup command asks before opening more than one copy, because every copy runs that command again.
- Profiles are saved on this PC only and never store passwords or keys. A profile keeps working even if you hide its shell from the plain list. If a profile points at a shell that is no longer detected (for example a WSL distribution you removed), its name is shown struck through and it will not launch until that shell is back.

Working with tabs

- Drag tabs to reorder them. Their identity stays stable, so reordering never disconnects a session.

- A remote tab whose connection is down (still connecting, dropped or failed) shows its label in red; use **Reconnect** to bring it back. Local shell tabs never show this.
- Switch tabs from the keyboard with **Alt+Right** (next tab) and **Alt+Left** (previous), even while a terminal is focused. The selection wraps around the ends, and plain arrow keys still do normal line-editing inside the shell.
- Opening a second tab for the same host adds a small number to the tab so you can tell duplicates apart.
- Use the **find box** to search the visible output of the active terminal. The match count is shown, and clearing the search removes the highlight.
- Maximize a single terminal to fill the window (also **F11**), and close a single tab or use **Close all** to clear them with a confirmation.
- Switch the pane area between **Tabs** and **Grid** with the toggle above the tab strip. **Tabs** shows one terminal at a time; **Grid** tiles every open terminal at once so you can watch several together. One terminal fills the area, two split it, and three or more tile two-across and scroll. Each grid tile has its own maximize and close buttons, its header shows the same colour as the tab, and clicking one makes it the active tab that find and shortcut commands act on. The active tile is also outlined in that host's own colour. Your choice is remembered between visits and restarts; maximizing a terminal always shows it on its own.
- The **Grid Titles** button (the eye, next to the **Grid** toggle) hides every tile's title bar for a denser, chrome-free wall of terminals; a crossed-out eye marks the hidden state and clicking again brings the bars back. It appears only in Grid and the choice is remembered across restarts.
- The grid tiles follow the order of the tabs above them. Drag a tab to a new position and the grid rearranges to match straight away, without disturbing any running terminal.
- The **Broadcast** button opens a rail down the left side: pick a shell language, tick the sessions you want, type a command at the bottom, and it is typed into all of them at once. Because you never leave the page, the panes you are watching are the ones that answer, which is the whole point of pairing it with **Grid**. The button remembers whether it was open across restarts.
- One shell language at a time, deliberately, so a command is always valid for everything it reaches. The safety checks are the same as on the **PTY Broadcast** tab: a destructive command asks for confirmation before going to SSH hosts, and every local-shell send is confirmed. That tab is still there and unchanged; it keeps the saved history and the full per-run report, while the rail is the quick, in-place version.
- **Fullscreen** (also **Ctrl+Shift+G**) gives the grid the whole screen: the toolbar, the tab strip, the sidebar and the status bar all slide away, leaving a thin strip at the very top. Move the mouse to that strip and everything slides back down, so you can pick a tab, close one, or use the dropdown without leaving fullscreen; move away and it hides again. **Ctrl+Shift+G**, **Esc**, or the button brings the chrome back. The shortcut is a Ctrl+Shift one on purpose: a terminal cannot tell Ctrl+Shift+G from Ctrl+G, so nothing is taken away from a program running in a tile, which a function key like F10 would have been.

- **Ctrl+Shift+T** puts the keyboard into the controls, which slides the strip down as it goes, and puts a ring around the button that has the focus. **Tab** and **Shift+Tab** then move along the toolbar and into the tab strip, where each tab's buttons show themselves as they take the focus instead of waiting for the mouse. Tab wraps round at the last control rather than walking off into the grid, and **Esc** hands the keyboard back to the terminal you came from (a second **Esc** then leaves fullscreen). Tab on its own cannot get you in here: a terminal has the focus in a grid, and Tab there belongs to the shell.
- Drag the right or bottom edge of any tile (or its bottom-right corner) to set the tile size. It sets the size for **every** tile, not just that one, so the grid stays even; tiles that no longer fit scroll rather than shrink. A dragged size lasts as long as Broadside is running and is then forgotten, and it does not change the **Grid** settings described below, which stay the one place a size is set for good. **Reset grid** in the toolbar goes back to tiles that fit the window, clearing both.
- That last part is the **Strict** checkbox in **Settings → Grid**, and it is on to start with. Turn it off if you would rather a drag counted: the tile width and height you drag out are written into the settings as you go, Settings shows the new numbers straight away, and the size is still there after a restart. Nothing else follows the drag either way — the column count, the layout, the tile headers and the side-by-side flip are yours to set and are never written by dragging.
- To take one terminal out of the grid, click the small **grid button** on its tab (it appears in Grid only). The tile disappears and the remaining tiles fill the space, while the shell keeps running untouched; the button turns amber so you can see which tabs are hidden, and clicking it again puts the tile back. Hidden tabs still work normally in **Tabs**.
- To pin the grid, open **Settings** and the **Grid** section. Leave everything on **Auto** for the layout above, or set the **Columns across** and the tile **Item width** / **Item height** in characters (columns and rows). Fixed tile widths that add up to more than the window scroll sideways rather than shrinking, so panes keep the character size you asked for. Changing the terminal font resizes pinned tiles to keep those dimensions. The settings apply the next time you open the Terminals page. One and two terminals are the exception: a single terminal fills the whole area and two take the full window side by side or stacked, both ignoring the Grid settings entirely, because at those counts a pinned size can only waste screen you could have been using.
- A pinned **Columns across** is trimmed to the number of terminals actually open, so six columns with three terminals up lays out three across the window rather than using half of it and leaving the rest empty. It follows as you open and close them, and it only ever narrows the count — a tile's pinned width or height is untouched and stays exactly what you set.
- When the terminals do not divide evenly into columns, the last row is short and something has to fill the gap. **Short last row** in **Settings → Grid** decides what: **Stretch the row** (the default) widens the tiles on it to take the whole width, so three tiles in two columns give two on top and one full-width underneath; **Tall tile in the last column** instead grows the tile above the gap down into it, giving one top left, one full height on the right and one bottom left; and **Leave the gap** keeps every tile the same shape and lets the empty cell stay empty. A size you

pinned wins over all of this: a stretched tile is wider than the **Item width** and a tall one is taller than the **Item height**, so where you have set that dimension by hand the gap simply stays.

Copy, paste and jumping to a tab

- **Copy** by selecting text with the mouse, and the selection is copied to the clipboard as soon as you release the button (no menu, no shortcut needed). A brief **Copied to clipboard** toast confirms it. You can also press **Ctrl+Shift+C** to copy the current selection.
- **Paste** with a **right-click** in the terminal, or with **Ctrl+Shift+V**. Plain **Ctrl+C** and **Ctrl+V** are left alone so the shell still receives them (for example Ctrl+C to interrupt a running command).
- **Text too small** is fixed from where you are reading it: hold **Ctrl** and turn the mouse wheel over a terminal to zoom the text in or out. Every open terminal changes together, one step a notch, between 8 and 32 px, and the size you stop on is saved as the terminal font size in **Settings** under **Appearance**.
- **Giving Ctrl and the wheel back**: while the zoom is on, that combination belongs to Broadside and the terminal never sees it. Two things go with it. The scrollback no longer scrolls faster when you hold **Ctrl**, and a full-screen program running with mouse reporting on (tmux with `mouse on`, vim with `set mouse=a`, htop) no longer receives the notch, so a mapping such as `<C-ScrollWheelUp>` stops firing. Little binds it by default, but if you rely on one, tick **Strict** beside the terminal font size in **Settings** under **Appearance**: the zoom stops, the size then only changes by being typed there, and Ctrl with the wheel goes back to the terminal exactly as it behaved before. The plain wheel, without Ctrl, is never taken either way.
- Every key Broadside takes, here and everywhere else, is listed in **Keyboard shortcuts**.
- With many tabs open, use the **Go to session…** picker at the top right to jump to any terminal. If its tab is scrolled out of view, the tab strip scrolls to bring it to the center so you do not have to hunt for it.

Full-screen prompts, and a stuck one

A tab is a real terminal, so the box-drawing prompts an upgrade puts up (a kernel restart notice, a config file question) work normally: arrow keys, **Tab** and **Enter** all reach them. Broadside claims no keys of its own beyond **Ctrl+F**, **Ctrl+Shift+C/V** and **Alt+Left/Right**.

If one of those boxes ignores everything you press, and your keystrokes appear on screen as `^[[A` or `^[` instead of moving the selection, the prompt is not listening: it was drawn by a program whose input is not this terminal, which is what happens when a script runs the upgrade rather than you. The keys are reaching the host and its shell is echoing them, so there is nothing to answer at that end. Press **Ctrl+C** to get back to a prompt, then run the command yourself, or run it non-interactively: `DEBIAN_FRONTEND=noninteractive NEEDRESTART_MODE=a` in front of an `apt` command stops both the config prompt and the restart prompt appearing at all.

Reset display in the toolbar redraws the active terminal from scratch, for a pane one of these left garbled. It clears that pane's screen and scrollback, then asks the shell to repaint its prompt (the same `Ctrl+L` you could press yourself), so you are not left looking at a blank pane

wondering whether the session is still there. Nothing is run and nothing is interrupted: it deliberately does not send `Ctrl+C`, so a long upgrade working away in that tab is safe from it. If a program has genuinely wedged the terminal, *you* still have to decide whether to interrupt it. A **Command Prompt** tab is cleared without the redraw, as `cmd` has no such key: press **Enter** there for a fresh prompt.

While a command is running in that tab there is no prompt to repaint, because the shell is not reading its keyboard until the command finishes. The pane simply clears and stays clear, and the prompt comes back on its own when the command is done, along with whatever the command printed while you were watching a blank pane. Broadside withdraws the redraw it asked for in that case, so it cannot arrive late and clear the screen a second time. If you had typed something ahead at that shell, it is left alone.

Shortcut commands

- The **Shortcut command** dropdown runs a saved command in the active tab. Each shortcut has a scope: **SSH / WSL (Linux)**, **Command Prompt / PowerShell**, or **Both** for commands that work everywhere (for example whoami).
- The dropdown shows only the shortcuts that run in the current tab, so a Linux command is never offered in a Windows shell. The icon next to each one reflects the active tab: the host icon on SSH and WSL tabs, the terminal icon on Command Prompt and PowerShell tabs.
- WSL tabs run Linux, so they use the SSH/Linux shortcuts (and show the host icon in the tab strip and the **plus button** menu). Add your own, with a scope, in **Settings** under Shortcut commands.

SECTION 4

Broadcast

Broadcast sends one command to many hosts at the same time and gathers each host's output in one place. It is the fastest way to run the same one-shot command everywhere.

How it runs

- Pick the hosts, type a command, and send. Each host runs it over an exec-channel and returns its output, which is grouped per host.
- Because the exec channel is not interactive, **Broadcast** is best for commands that finish on their own and print a result. For anything that prompts you or needs a live screen, use a **Terminals** tab or **PTY Broadcast** instead.
- A per-command time limit stops hosts that take too long. The output area keeps your history and follows new output as it arrives, and you can hide the per-host headers if you prefer a compact view.

Choosing and filtering hosts

- Pick targets in the host rail on the left. It collapses to color dots with the arrow button, has a **Sort** dropdown and a **Select all** box, and its collapse state is remembered. No hosts are selected by default; you choose the targets for each broadcast.
- Two filters narrow the rail: a **Find by label…** box that matches host labels as you type (like a find bar), and a **Filter tags** dropdown that works exactly like the tag filter on the **Hosts** table: uncheck tags (or the untagged bucket) to hide hosts, with **All** / **None** shortcuts. The two combine, and each filter lasts for the session and resets on restart.
- Filtering only ever narrows what you can act on: **a host the filter hides is unchecked automatically**, and it comes back *unchecked* when you clear the filter, so a hidden host can never be swept into a broadcast. **Select all** and the counter apply to the visible hosts only. This same rail filter is on **PTY Broadcast**, **MultiTerminal** and **SFTP Broadcast**.

Safety

A destructive-command guard can warn you before sending commands that look dangerous, and ask you to confirm first. You configure which patterns count as destructive in **Settings**.

SECTION 5

PTY Broadcast

PTY Broadcast types one command into every open interactive terminal at once. Unlike plain **Broadcast**, it drives the live shells you already have running in the **Terminals** tab.

When to use it

- Use it when you want the same keystrokes delivered into several live sessions, for example to start the same interactive program on many hosts or local shells.
- Because it sends into a live PTY, there is no completion signal and no per-command timer: you are typing into real shells, exactly as if you typed in each tab yourself.
- Plain **Broadcast** runs a command and waits for a result. **PTY Broadcast** just sends the text. Choose **Broadcast** for one-shot commands and **PTY Broadcast** for interactive ones.

Four tabs, one shell language each

- The page is split into four fixed tabs: **SSH**, **PowerShell**, **Command Prompt**, and **WSL**. Each open terminal shows up under its own tab, so a single command is always valid for the shells it reaches (PowerShell and Command Prompt stay separate because their syntax differs). A small count on each tab shows how many shells of that kind are open.
- The **SSH** tab is the host-oriented view: it has the same **Find by label…** box and **Filter tags** dropdown as **Broadcast** (filtering a session out unchecks it), applied to each session's host, plus the persisted dispatch history.

Local shells (PowerShell, Command Prompt, WSL)

- The three local tabs fan a command out to the shells running on *this machine*. A persistent yellow banner marks the danger: the command runs in every checked shell immediately, with no undo.
- Every local send asks you to confirm first, listing the exact command and the shells it will run in. Cancelling sends nothing.
- Local runs are session-only: the report clears when you close Broadside (local shells do not survive a restart anyway). Behind the scenes local shells receive a carriage return so PowerShell and Command Prompt actually execute the line rather than waiting at a continuation prompt.

SECTION 6

MultiTerminal

MultiTerminal runs a command across the terminals you choose and shows every host's output together, each tinted with that host's color so you can compare results at a glance.

How it works

- Pick which open sessions to target in the host rail on the left, type a command in the composer, and send. You need at least two terminals open for the combined view to be meaningful. The rail collapses to color dots; its actions shrink to icons when collapsed.
- The rail carries the same **Find by label…** box and **Filter tags** dropdown as **Broadcast**. Only connected, unfiltered sessions are selectable, and hiding a session with the filter unchecks it.
- Each command produces a block in the log, labeled with the command that ran. You can delete an individual block to keep the view tidy.
- Turn on **Show Manual Activity** to also capture commands you type by hand inside a **Terminals** tab; with it off, only commands sent from **MultiTerminal** appear here. It follows the hosts you have checked in the rail, so a host you deliberately left unchecked stays out of the log whether you type into it or not.

Block headers

- With **Headers** on, each block has a header you can click to collapse or expand its output. Hover the header for a **copy** icon (copies that block's output) and, when the host still has an open terminal, a **terminal** icon that jumps to its tab.
- With **Headers** off, blocks show output only, with no header to collapse, copy or jump from.

Rail actions

- **Clear results** empties the block log (and the saved history). **Clear command history** clears the Up/Down recall shared with **Broadcast**. **Close all terminals**tears down every open terminal after a confirmation.

SECTION 7

Skills

A **skill** is a saved sequence of steps that Broadside runs against every host you check, in a real interactive shell it opens for each one. Because it is a real shell rather than a one-shot command, a skill can become root and stay root, answer questions a program asks, and drive things that repaint the screen, while you watch each host's terminal live.

The five kinds of step

- **run** sends a command and waits for it to finish, then branches on whether it succeeded.
- **wait for** watches the output for a pattern, optionally sending a reply the moment it appears. This is how you answer a prompt: wait for `Password:`, send the password.
- **send** types something with no waiting, for a keystroke a program is already sitting there expecting.
- **hold** pauses for a fixed number of seconds, sending nothing, so a status screen has time to settle.
- **post to the skill log** writes a line into that host's run log and carries on. It touches the host in no way at all; it is the skill explaining itself.

A skill that explains itself

Post to the skill log is for a skill that does different things depending on what it finds. The branch it took is obvious to the skill and invisible to whoever reads the run afterwards, so let it say so: *already on the current kernel, nothing to do*. Pick a **Log type** of **Note**, **Warning** or **Error**, write the message, and say where to go next. Inputs work in the message, so it can name what it found.

One combination does more than log. An **Error** whose **Then** is **stop** ends that host as a failure carrying your message, which is how a skill refuses a host in its own words: *cancelled itself, this is not a web server* reads as a failure with a reason instead of a green tick for a skill that did nothing. An **Error** that goes anywhere else is just a loud log line, so a skill can complain and still clean up after itself.

A log step's message is one line in the run log, tinted by its type, and that line is also the step's own entry: it is not announced and then repeated.

Patterns

A **wait for** pattern is a regular expression tested against the output with the colour codes stripped out. Its anchors are line anchors, so `^Ready$` means a line that says Ready, not the whole screen.

Interactive steps

Normally a **run** step knows the command finished because the shell comes back to a prompt. Some commands never do: `sudo -i` opens a nested shell, and an installer stops on a question. Tick **Interactive** on those, and the step moves on once the output goes quiet instead. You then drive the rest with **wait for** and **send**.

Branching

- Every step says where to go next. **next** means the following step in the list, so a straight-through skill needs no wiring at all. **stop** finishes that host successfully.
- A **run** step branches on the command's exit code by default: one target if it worked, another if it did not.
- Tick **branch on the output** instead to decide on what the command printed. Give it a pattern and two targets, and the output test outranks the exit code entirely. Useful for a command that reports what it found without failing, such as `systemctl is-active`.

Timeouts

Each waiting step has a time budget. When it runs out the default is to **pause and wait for you**: that host goes yellow and you choose to wait again, treat the step as done and carry on, or stop the host. Nothing is ever quietly reported as finished because a timeout expired.

The other two options are to fail (which ends the host for a **Wait for output** step, or takes the **if it fails** branch for a **Run a command** step), or **go to a step**, which carries the run on somewhere else of your choosing. Picking it shows the same target list the other branches use, so a step that times out can take a recovery path, retry an earlier step, or finish cleanly instead of stopping the host. It also lets you handle a slow command differently from one that failed, which the **if it fails** branch alone could not tell apart. The timeout branch is drawn on the flow map like any other, so a loop it creates shows up in the usual warnings.

Inputs

Declare inputs to avoid hard-coding values into commands. Write `{{service}}` in a step and Broadside asks you for **service** just before the run. Go templates such as `docker ps --format '{{.Names}}'` pass through untouched, so they are safe to use.

The flow map

- Clicking a skill's name opens its **overview**: what it does, the inputs it asks for, and a diagram of how its steps connect. Clicking the pencil skips straight to the editor instead.
- Click any step in the diagram to see its settings. The zoom controls sit at the top right; clicking the percentage returns to 100%.
- The map catches two mistakes a list of steps cannot show you: a step nothing branches to, which will never run, and a loop with no way out, which will run into the repeat cap and fail the host. Both appear as amber warnings above the diagram. Neither blocks you, as a loop with an exit branch is a perfectly good pattern.

- A loop is drawn as a dashed amber arrow that runs back up the left of the diagram, so it reads as going backwards rather than being mistaken for another branch forward.

Runaway loops

A branch that loops is a legitimate way to poll until something is ready, so Broadside caps it rather than refusing it. **Stop a host after a step runs N times** in the editor sets the budget: any single step reaching it stops that host with a message naming the step. It counts per step, not per run, so it is the retry budget for the step that loops rather than a limit on how long a skill may be. The default is 100, which is also the ceiling.

Let steps repeat without limit removes the cap altogether, and it is dangerous: use it only if you truly know what you are doing. With no cap, a skill whose branches loop never finishes on its own. It keeps running steps against every host it is on until you stop it, and a loop that writes files, starts processes or installs things can fill a disk or exhaust memory on the far side while every host it touched holds an open shell. **Stop host** and the **Emergency stop** still work and are the only things that will end it. It is off by default, it is a switch of its own rather than a number you can mistype into, the flow map's loop warnings say so, and the run confirmation says so again before you dispatch. The legitimate use is polling something with no bound, watched.

Running one

- Check hosts in the rail, then press play on the skill. Steps that match a **guard rule** need the same typed confirmation a dangerous broadcast does.
- Each host gets its own live terminal pane. A host that pauses shows its options there. **Stop host** ends one host and leaves the rest running; the **Emergency stop** button at the foot of the rail ends all of them at once, which can leave a host mid-upgrade.
- You can type into any host's pane while it runs. That takes the host over: the skill stops driving it and it is yours.
- Each pane's header names the step it is on and where that sits in the skill, next to the countdown, so a long wait still tells you what is being waited for.
- **Log** in a pane's header shows what the skill decided for that host: each step it started, what matched, what it sent, what timed out and which branch it took, along with anything a **post to the skill log** step had to say. The terminal beside it shows what the host said; this is the half that explains why the run went where it did.
- When the run is over, **Rerun** beside **Close run** starts the same skill again. It closes this run first (shells and all) and takes you back to the confirmation, so the hosts and inputs are checked rather than silently reused. Panes you had the log open on open it again.

Keeping the shell afterwards

A skill normally closes each shell when it is done. Turn on **Allow transfer** and the shell stays open instead, so you can press **To terminal** and carry on by hand in a **Terminals** tab, with the run's scrollback intact. Broadside checks whether that shell is root and warns you before you leave a privileged one open. Closing the run closes any it still holds.

Building one

- Steps in the editor are folded down to one line each, showing the id, the kind and what the step does. Click a step's header to open it, or use **Expand all** to open the lot. A step you add opens ready to fill in, and the editor scrolls down to it.
- The step you last clicked is tinted. That is mostly so it stays findable while you work, but it earns its keep with the up and down arrows: the tint travels with the card as it moves, so you can see what you just moved and where it went.
- **Renumber** renames the ids to s1, s2, s3 in their current order and rewrites every branch to match, so the flow is unchanged. Use it to tidy up after reordering.
- Amber notes below the steps are advice, not errors: they never stop you saving or running. Only the red list does that.

Organising them

- The arrows on each skill in the rail move it up and down the list. The order is yours and it is remembered.
- **Clone** starts a new skill from a copy of an existing one, named **(clone)**, which is the way to build a variant of something that works. It opens in the editor unsaved, so nothing is written until you press **Create skill**, and the original is untouched whatever you do next.
- **Export** writes a skill to a `.json` file, and **Import** reads one back, showing you its steps before anything is saved. Files carry the definition only, never credentials.
- A skill file records which version of Broadside it needs, and only when it needs one: a plain skill still imports into an older copy of the app. One that uses something newer (a log step, a timeout that goes to a step, a removed repeat cap) is refused there with *made by a newer Broadside*, rather than a parse error that reads as a corrupt file. If you see that, the app doing the importing is the one to update.

What a skill needs from the host

Skills need a **bash**, **zsh** or **sh** login shell. A host on fish or csh is refused up front and named on the **Hosts** tab with an amber chip.

SECTION 8

SFTP

SFTP browses and moves files over the same secure SSH connection as the rest of the app (see sftp). It has two tabs: **Commander**, a two-pane file manager for a single host, and **Broadcast**, which sends or pulls one path across many hosts at once.

Commander (one host)

- A two-pane view: your PC on the left, the remote host on the right. Open folders to navigate, and use the path bar to jump around; both sides show the same columns so they read alike.
- Transfer by dragging an item from one side to the other, or with the transfer buttons. Folders copy recursively, with a progress bar and a running byte count.
- Select several items to move at once, the same way Windows Explorer works: **Ctrl+click** to add or remove individual files, and **Shift+click** to select a whole range between two rows. A plain click selects a single file (or opens a folder). Dragging any selected item then transfers the whole selection in one go.
- You can make a new folder on either side and delete items. Local deletes go to the Windows **Recycle Bin** so they are recoverable; a non-empty remote folder is refused with a clear message rather than failing silently.

Broadcast (many hosts)

- The **PUT / GET** toggle sets the direction: **PUT** sends one local file or folder to every selected host; **GET** pulls one remote path from every host into its own subfolder on your PC (one folder per host label, so files from different hosts never collide). Each host gets its own progress bar, and a few hosts transfer at a time so many targets are queued rather than opened all at once.
- On **GET**, the remote path may end in a wildcard (***** or **?**) to pull several files at once, for example **/home/user/*.log** fetches every matching file from each host. Only the last part of the path is matched; the folders leading up to it are taken literally.
- Because a broadcast write can overwrite data on many machines at once, it is gated: an amber banner explains the risk and you must type **CONFIRM** before *every* run. By default no hosts are selected, and the rail has the same tag + label filter as **Broadcast**.
- **Create path if it doesn't exist** (PUT only) makes the destination directory on each host when it is missing; it is a session-only toggle that resets on restart.
- First contact with a host is verified with TOFU just like elsewhere: unknown keys are gathered into one **trust** dialog after the run, and trusting them retries those hosts. A host whose key has *changed* is refused, never offered for trust.

When a file already exists

The selector on the tab row applies to *both* tabs: **Overwrite all** always replaces, **Newer only** replaces only when the source is newer, and **Skip existing** leaves same-named files untouched. It governs recursive folder transfers (a single file always overwrites), is remembered across

> restarts, and so is the tab (Commander or Broadcast) you last had open.

SECTION 9

Logs

The **Logs** tab is your record of what happened: saved terminal sessions, an audit log of security-relevant actions, and your past commands.

> **What you can find**
>
> - Saved sessions let you reopen and read the captured output of an earlier terminal session.
> - The audit log records important actions over time. You can switch it between compact and pretty-printed views, and turn audit logging on or off in **Settings**.
> - Command history lists commands you have run, and every view has a search box.

SECTION 10

Settings

Settings is where you tune how Broadside behaves. Use the search box at the top to jump straight to a section.

> **What you can change**
>
> - **Performance**: how many hosts the app talks to at once.
> - **Network probe**: how the app checks whether hosts are reachable for the **Status** column.
> - **Destructive command guard**: the patterns that trigger a confirmation before a risky command is sent.
> - **Shortcut commands**: reusable one-click commands for the dropdown on the Terminals and Broadcast pages. Each one is scoped when you add it: **SSH / WSL (Linux)** commands run on SSH hosts and WSL tabs, **Command Prompt / PowerShell** commands run on local Windows shells, and **Both** commands run everywhere. See the Terminals section for how the dropdown uses the scope.
> - **Appearance**: light or dark theme, the terminal font and its size, the size of the application text, which **Hosts** columns to show, and which detected local shells appear in the Terminals **plus button** menu. Holding **Ctrl** and turning the mouse wheel over any terminal changes the terminal font size from where you are reading and saves it here, unless **Strict** beside that field is ticked, which leaves the wheel alone.
> - **Backup & Restore**: save a copy of your app data, or restore from one. Backups contain your host list and settings but never your passwords or key passphrases (those stay in the **Security section** vault), so a backup file cannot leak a secret. Tick **Also include hosts CSV** to additionally write a plain spreadsheet of your hosts alongside the backup.
> - **Audit log**: turn audit recording on or off.
> - **Security**: the sudo auto-fill switch and the admin lock, both described in the Security section below.
> - **Reset**: return every setting to its default, behind a confirmation.
> - **Danger Zone**: **Delete all hosts & credentials** removes every host and its stored secrets in one step. It cannot be undone, so the dialog offers to back up first (optionally including the hosts CSV) before it wipes anything.

SECTION 11

Security

Broadside handles passwords and connects to your machines, so it is worth understanding exactly what it stores, what it sends, and the controls you have. **There is no telemetry and the app never phones home.**

How your credentials are stored

- Secrets (SSH passwords, key passphrases and sudo passwords) are stored in the credential-manager first, which ties them to your Windows login. If that is unavailable, the app falls back to a file named credentials.age that is encrypted at rest.
- Each secret is keyed to its host, so removing a host removes its secrets. Non-secret host details (label, address, port, username, color) live in a local database in plain text.
- Once saved, a secret is never shown again in the app. You can replace it, but the stored value is never displayed.

What is sent over the network

- The app makes outbound SSH connections only to the hosts you configure. **Broadcast** uses an exec-channel; **Terminals** use a full PTY. Nothing else is sent anywhere.
- Host identity is verified with TOFU: the app remembers each host's key on first connection and warns you if it changes.
- When the app fills a sudo prompt, the password is sent inside the encrypted SSH session in answer to the prompt. It is never placed on the command line or echoed back to the screen.

The sudo auto-fill switch and admin lock

- Sudo auto-fill is a global switch in **Settings**. When on, the app answers a host's sudo password prompt using the password you stored for that host. Turn it off to always type sudo passwords yourself.
- It answers a **sudo** prompt and nothing else. A password prompt from **chsh**, **su**, an inner **ssh** or a database client looks much the same on screen, but it is a different program asking, sometimes on a different machine, and a stored password typed into it would have travelled somewhere you never chose to send it. Those prompts are yours to answer.
- The admin lock is an optional passcode that protects the credential and security controls. It is for authorization only: it uses argon2 and encrypts nothing, so losing the passcode loses no data. Setting it gives you a one-time recovery code to regain access if you forget the passcode.
- The admin lock is not a wall between separate people on one Windows account. The real boundary for limiting a less-trusted operator is to give them their own Windows login and have them import hosts from a credential-free csv file, so they never receive your stored secrets.

SECTION 12

System requirements

Broadside is a native Windows desktop app and runs comfortably on modest hardware. The table lists what it needs at a minimum, and what makes for a smoother ride when you are driving many hosts at once.

Minimum and recommended

Component	Minimum	Recommended
Operating system	Windows 10 (64-bit), version 1809 or later	Windows 11 (64-bit), kept up to date
WebView2 runtime	Microsoft Edge WebView2 installed (the installer adds it if missing)	WebView2 kept current through Windows Update
Processor	64-bit dual-core (x64 or ARM64)	Quad-core or better, 2.5 GHz and up
Memory (RAM)	4 GB	8 GB or more
Free disk space	300 MB	1 GB or more, for logs, saved sessions and backups
Display	1280 × 720	1920 × 1080 or higher
Network	Any connection that can reach your SSH hosts	A low-latency link when broadcasting to many hosts at once

Worth knowing

- Broadside is Windows only: it uses ConPTY for local shells and the credential-manager for your secrets, neither of which exists on macOS or Linux.
- No administrator rights are needed to run it. WebView2 ships with Windows 11 and recent Windows 10; on an older system the installer adds the Evergreen runtime, which is what draws the app's interface.
- Memory and processor use scale with how many terminals you keep open and the concurrency you set for **Broadcast** and **Skills** in **Settings**; each terminal also keeps its own scrollback in memory.
- Disk use grows with saved sessions, the audit log and backups, all of which you control from the Logs and Settings tabs.

If Windows Security flags Broadside

- It can happen, and it is a false alarm. An application that is not yet code-signed, is only days old, has few users, and opens shell sessions matches the pattern Windows Defender's automatic heuristics are built to be suspicious of. Opening terminals means starting processes, which is the behaviour being scored, and it is also the entire purpose of the app.

- A detection name ending in `!ml` means no known malware was matched at all: a model simply judged the file to look unusual. Broadside has been flagged this way once, as `Trojan:Win32/Bearfoos.A!ml`.
- If the app disappears, restore it from **Windows Security** under **Virus & threat protection** and **Protection history**. Before you do, check the installer's SHA-256 against the one published with the release, so you know you are restoring the file that was actually built.
- Please report it if it happens, with the detection name and version. Each report goes to Microsoft as a false positive, which is what gets the detection corrected for everyone. Adding a Defender exclusion silences the warnings but also stops Defender watching that folder, so treat it as a temporary measure rather than a solution.

SECTION 13

Keyboard shortcuts

Every key Broadside takes for itself, and what it does. Anything not on this list belongs to whatever has the focus — in a terminal that means the shell, which is why the shortcuts here are nearly all **Ctrl+Shift** chords: a terminal cannot see the Shift, so **Ctrl+Shift+G** costs a program running in a tile nothing at all, while a function key would have taken **F10** away from `mc` and `nano`.

Key	Where	What it does
ANYWHERE IN BROADSIDE		
`F11`	Terminals	Maximize the selected terminal to fill the window, hiding every other part of the app. Press it again to restore. It works from inside a maximized terminal wherever you are.
`Ctrl+F`	Terminals, Broadcast, Logs	Open the find bar for whatever output you are looking at, instead of sending Ctrl+F to the shell.
`Enter`	Find bar	Go to the next match.
`Esc`	Find bar, dialogs, menus	Close it without doing anything.
THE TERMINALS GRID		
`Ctrl+Shift+G`	Grid	Fill the screen with the grid: the sidebar, status bar, toolbar and tab strip all go, and the tiles take the whole window. Press it again to bring them back. Deliberately not a function key — F10 belongs to mc and nano.
`Esc`	Grid fullscreen	Leave fullscreen. With the keyboard in the controls strip it hands focus back to the terminal instead, and the next press leaves fullscreen.
`Ctrl+Shift+T`	Terminals	Put the keyboard into the controls. In fullscreen the hidden strip slides down and its first button takes focus, with a ring around it so you can see where you are. This is the only way in: Tab inside a tile belongs to the shell.
`Tab` / `Shift+Tab`	Terminal controls	Walk forwards and backwards along the toolbar and tab strip once Ctrl+Shift+T has put you there. The strip stays down the whole time, and Tab wraps round at the last control rather than dropping into the grid. Esc is the way out.
`Alt+Left` / `Alt+Right`	Terminals	Previous / next terminal tab, wrapping at the ends. Works with the focus inside a terminal — the shell never sees the key.
INSIDE A TERMINAL		
`Ctrl+Shift+C`	Terminals	Copy the selection. A terminal cannot see the Shift, so plain Ctrl+C is untouched and still interrupts what is running.

Key	Where	What it does
`Ctrl+Shift+V`	Terminals	Paste the clipboard into the shell as if typed. Plain Ctrl+V is left to the shell.
`Right-click`	Terminals	Paste, the PuTTY and Windows Terminal way. No context menu appears.
`Select with the mouse`	Terminals	Copies as soon as you let go of the button, with a short toast to confirm it. There is no separate copy step.
`Ctrl+mouse wheel`	Terminals, Skills	Make the terminal text bigger or smaller, one step a notch, between 8 and 32 px. Every open terminal follows, and the size you stop on is saved in Settings, Appearance. Broadside takes this combination while the zoom is on, so the terminal does not see it: tick Strict beside that field to give it back to tmux, vim, htop and the fast scrollback.
`Tab`	Terminals	The shell's own completion, always. Nothing in Broadside takes it.
RENAMING A TAB		
`Enter`	Tab rename	Commit the new name. An empty name clears the override and the tab goes back to its shell name.
`Esc`	Tab rename	Cancel and keep the old name.
THE COMMAND COMPOSER (BROADCAST, PTY BROADCAST, MULTITERMINAL)		
`Enter`	Composer	Send the command.
`Shift+Enter`	Composer	Start a new line instead of sending, for a multi-line command.
`Up` / `Down`	Composer	Walk back and forth through the commands you have sent. Only from the first / last line, so editing a multi-line command still moves the caret normally. What you had typed is kept as a stop on the cycle, so you can always get back to it.
SFTP COMMANDER		
`Ctrl+click`	File list	Add or remove one file from the selection.
`Shift+click`	File list	Select everything between the last click and this one.
THIS HELP PAGE		
`Type in the filter`	Help	Searches everything a section says, including these tables, and keeps the ones that match: if you can read it here you can find it here. Matching words are marked in amber and the first is scrolled to, with arrows beside the count for the rest. Punctuation is optional: typing ctrl shift g finds Ctrl+Shift+G, and 16 20 finds 16/20. Each section in the list on the left carries the number of marks inside it. A section can also be found by words it does not use, such as the wrong name for a thing; it is kept with nothing marked in it and the list says keyword instead of a number.

SECTION 14

Warnings and messages

What Broadside can put in front of you, and what each one actually means. Warnings name the thing that went wrong rather than the thing you should do about it, so this is the other half.

What you see	Where	What it means
ON A HOST ROW		
An amber chip naming a shell (fish, csh, tcsh, …)	Hosts	This host's login shell is not one of the Bourne-family shells Broadside can drive. Terminals, broadcast and file transfer all work as normal; command block tracking is unavailable and skills cannot run here. Sudo password auto-fill still works. The chip names whatever the host reported, and Broadside only reads a name it can recognise as a shell path — a chip naming something from your own startup script was a bug, fixed in 1.3.0. The chip is what the last completed connection read: if it could not read one at all the chip goes, and upgrading to 1.3.0 clears any name that could never have been a shell.
No credentials set	Hosts	The host is saved but has no password or key stored, so nothing can connect to it. Edit the host and add one.
A latency in milliseconds after a test	Hosts	Not a warning. The connection was made, authenticated and closed again, and that is how long connect and auth took.
OPENING A TERMINAL		
Unreachable: …	Terminal pane	Broadside never got as far as a login. The rest of the line is the operating system's own words for why. Try again is beside Close tab, because most of these are worth a second attempt.
Unreachable: An existing connection was forcibly closed by the remote host. (os error 10054)	Terminal pane	The server accepted the connection and then hung up during the handshake. Most often that is OpenSSH's MaxStartups: it counts connections that have not finished authenticating, and past ten (the default) it starts dropping them — which is what opening a wall of terminals at once looks like from the server's side. Broadside now keeps at most eight handshakes in flight and retries a dropped one twice, so this should be rare; if it persists, raise MaxStartups on the host or open fewer at a time.
Unreachable: connection timed out after 10s	Terminal pane	Nothing answered on that address and port. The host is off, the port is wrong, or a firewall is dropping it. Not retried automatically — a host that is not there does not become there.
Authentication failed: …	Terminal pane	The host answered and refused the credentials. Wrong password, wrong key, wrong username, or the key needs a passphrase Broadside was not given.
No credentials stored. Edit the host on the Hosts page.	Terminal pane, Broadcast	Nothing was tried: there is no password or key saved for this host.

What you see	Where	What it means
Waiting on host-key verification…	Terminal pane	The connection is held open while you answer the host-key dialog. Nothing is sent to the host until you do.
HOST KEYS		
This host's key is not known yet	Host-key dialog	First connection to this address and port. Broadside has no key on record, so it is showing you the fingerprint the server presented and asking whether to trust it. Check it against the host itself before accepting.
The host key has changed — the connection was refused	Host-key dialog	The server presented a different key from the one on record. That is either a rebuilt or re-keyed server, or the connection being intercepted. Nothing was sent and no password was offered. Do not accept it until you know which.
host key changed — refused	SFTP Broadcast	The same thing, on one host of a batch. The others carried on.
WHILE A TERMINAL IS OPEN		
A slim banner across the bottom of a tile	Terminal pane	The shell has exited or the connection dropped. The scrollback is kept and stays readable; Reconnect starts a fresh shell on the same host, Close tab throws it away.
Every terminal is hidden from the grid.	Grid	Not a failure. Terminals are open but every one has been taken out of the grid from its tab. Use the grid button on a tab to bring one back.

Anything that reaches the Logs tab is recorded with the host it happened on and the operation that was running, which is the place to look when a message went by too quickly to read.

SECTION 15

Glossary

Plain-language definitions for the technical terms used above. Links throughout the documentation jump here.

SSH (Secure Shell)

A network protocol for running commands on another computer over an encrypted connection. Broadside uses SSH to reach every host you add. All traffic between the app and a host travels inside this encrypted channel.

SFTP (SSH File Transfer Protocol)

A way to browse and copy files over an SSH connection, using the same login and encrypted channel as a remote shell. Broadside's SFTP tab uses it to move files to and from your hosts; nothing travels outside the SSH session.

PTY (pseudo-terminal)

A two-way text channel that behaves like a real terminal screen and keyboard. Interactive programs (a login shell, a text editor, a progress bar) need a PTY so they can redraw the screen and read keystrokes. Each Terminals tab opens its own PTY.

ConPTY (Windows pseudo-console)

Windows' built-in version of a PTY. Broadside uses it to run local Windows shells (PowerShell, Command Prompt, WSL) inside a Terminals tab, the same way it runs remote shells over SSH.

Shell

The program that reads the commands you type and runs them, for example bash on Linux or PowerShell on Windows. A terminal tab is a window onto a shell.

POSIX

The standard that Unix-like systems agree on, and the reason a command typed for Linux usually works on macOS or inside WSL. In Broadside it is the name of a shell family: a Bash-like terminal such as Git Bash, MSYS2 or Cygwin speaks the same command language as a Linux host, so those tabs are grouped with WSL rather than with PowerShell or Command Prompt. A custom terminal set to the Bash-like language is shown with a `posix` tag and joins the **WSL / POSIX** tab in **PTY Broadcast**, which keeps a broadcast command valid for everything it reaches.

sudo

A Unix command that runs another command with administrator rights. It usually asks for your password first. Broadside can fill that password prompt for you when the feature is turned on (see the Security section).

TOFU and host keys

Every SSH server has a unique host key, a fingerprint that proves it is the same machine you connected to before. TOFU (Trust On First Use) means the app remembers a host's key the first time you connect and warns you if it ever changes, which can signal that the connection is being intercepted.

SSH key vs password

Two ways to prove who you are to an SSH server. A password is a secret you type. An SSH key is a pair of files (a private key you keep and a public key the server holds); the private key can be protected by a passphrase. Keys are generally more secure than passwords.

Key passphrase

An optional secret that unlocks an SSH private key file, so that simply copying the file is not enough to use it. Broadside can store the passphrase for you in the same protected place as your other secrets.

Windows Credential Manager

A secrets vault built into Windows, tied to your Windows user account. Broadside stores passwords and passphrases here first so they are protected by your Windows login and never kept in plain text.

argon2

A modern password-hashing algorithm. Hashing turns a passcode into a fixed fingerprint that cannot be reversed back into the original. The admin lock stores only this fingerprint, so the passcode itself is never saved anywhere.

CSV

A plain-text spreadsheet format (comma-separated values) where each line is one record. Broadside can import a list of hosts from a CSV file. The importer never reads passwords from the file.

Exec channel

A way to run a single command on a host over SSH and collect its output, without opening a full interactive screen. Broadcast uses this. It is faster for one-shot commands but cannot drive interactive programs.

ANSI and xterm

ANSI codes are the invisible control sequences a program sends to color text, move the cursor and clear the screen. xterm is the long-standing terminal standard those codes follow. Broadside renders them so remote output looks the way it would in a native terminal.